SUMMER INTERNSHIP REPORT

on

AI Accelerator for Tinyml using RISC-V Architecture and FPGA

Duration: June 22, 2026 to July 31, 2026

Under Supervision Of

Prof. Amit Prakash Singh
USIC&T, GGSIPU, New Delhi

Mrs. Ishu
USIC&T, GGSIPU, New Delhi

Submitted By

Name: Shashank Pandey
Enrolment no.: 08916412824
ECE-2, USI&CT

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY
NEW DELHI

AICTE IDEA Lab–Guru Gobind Singh Indraprastha University
E-Block, Guru Gobind Singh Indraprastha University Sector-16C, Dwarka, New Delhi–110078

July, 2026

Certificate

This is to certify that the internship project entitled "**AI Accelerator for Tinyml using RISC-V Architecture and FPGA**" was successfully completed by the following students from various colleges in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) under the internship program at AICTE IDEA Lab–Guru Gobind Singh Indraprastha University, New Delhi–110078.

S.No.	Name	Institute
1.	Shashank Pandey	USI&CT, GGSIPU
2.	Nitin Kr. Shukla	USI&CT, GGSIPU
3.	Yash	USI&CT, GGSIPU
4.	Nitin	USI&CT, GGSIPU
5.	Rajneesh Kumar	USI&CT, GGSIPU

Supervisor
Prof. Amit Prakash Singh
USIC&T
GGSIPU, New Delhi

Mentor
Mrs. Ishu
USIC&T
GGSIPU, New Delhi

Declaration

We hereby declare that the project work presented in this internship report, entitled “AI Accelerator for Tinyml using RISC-V Architecture and FPGA” is entirely our own work and has not been submitted for any degree or diploma from this or any other institute for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech).

Name	Institute
Shashank Pandey	USI&CT, GGSIPU
Nitin Kr. Shukla	USI&CT, GGSIPU
Yash	USI&CT, GGSIPU
Nitin	USI&CT, GGSIPU
Rajneesh Kumar	USI&CT, GGSIPU

Acknowledgment

I would like to express my gratitude to my college,
University School of Information, Communication & Technology, Guru Gobind Singh Indraprastha University, for providing me with the opportunity to participate in this internship program during the summer. Their encouragement has been of great significance to my development.

I would also like to express my gratitude to AICTE IDEA Lab–Guru Gobind Singh Indraprastha University, E-Block, Guru Gobind Singh Indraprastha University, Sector-16C, Dwarka, New Delhi–110078, for arranging the internship and providing me the opportunity to take part in it. The knowledge and experience that I have obtained from my time spent here have been really beneficial.

I would like to express my gratitude to my mentor, Mrs. Ishu, for providing me with direction, support, and constructive criticism. I would not have been able to complete this assignment without your assistance.

I would also like to express my gratitude to everyone else who assisted me throughout this internship. For their assistance, I would like to extend my special gratitude to Prof. Amit Prakash Singh for providing all the required resources and valuable guidance. Your assistance has been of immense value.

Last but not least, I am incredibly grateful to all the other faculty members who have contributed significantly to my overall education.

Abstract

In today's world, there are machines in the factories which rotate at very high speeds. Such rotating machines have one critical component called a bearing. The bearings get worn out with time, become exhausted, and they may fail. In case of failure of the bearing, the machine would stop working all of a sudden, resulting in the loss of huge sums of money for the factory. In order to avoid such a disaster, there should be a way to hear the voice of the machine before it fails.

During this summer project, we built a smart, automatic system to check if a machine bearing is healthy or broken. We did this by carefully recording how the machine shakes while it spins. We used a small computer brain, which is also known as a neural network, to look at the shaking patterns and make a smart guess. Artificial Neural Networks (ANNs) have proven effective in establishing associations between input features and target classes, though their floating-point implementations can be resource-intensive.

Two very distinct implementations of this brain have been performed by us. The first implementation was through the use of a regular CPU. This is known as implementing the brain through "software". A regular CPU works sequentially, performing calculations one after another. It is extremely precise but relatively slow. The second implementation was the very aim of our whole project. We created an entirely new, unique custom chip from scratch. The custom chip was created using an FPGA board. FPGA is a special board which allows connecting digital blocks to each other to create your own hardware physically. FPGAs are highly valued because of their reconfigurability, customizability, energy efficiency, and flexible architectural design.

We designed this custom chip to do only one specific job: run the computer brain. Because it only does one job, it does not get distracted, and it runs extremely fast. We ran tests to compare the normal software processor against our special custom hardware chip. The results were amazing. We found that our custom hardware chip is about 147.5 times faster on average than the normal software processor.
Most importantly, both the normal software chip and our super-fast custom chip gave the exact same correct answers every single time. This report explains how we built this entire system step by step.

List of Figures

Table of content

Contents

1. Introduction

Artificial Intelligence (AI) and Feed-Forward Neural Networks (FFNNs) have been widely adopted to perform classification and prediction for industrial purposes, such as in the field of machine health. Nevertheless, performing such tasks in software can be highly power-consuming and have great latency. The presented project implements an FPGA-based FFNN accelerator that was written in RTL Verilog for the PYNQ-Z2 board.

The proposed solution was built using AMD Vivado and provides hardware units for accelerated neural network inference. As a result, low-latency, power-efficient, and reliable computations were achieved, enabling the real-time implementation of edge-based predictive maintenance models.

1.1 Background

Modern-day industries are increasingly adopting intelligent machine health monitoring systems that utilize sensor readings, such as vibration and temperature measurements, for early fault detection and prediction. Feed-Forward Neural Networks (FFNN) have been demonstrated to perform such diagnostics tasks with a high level of accuracy. However, software-based inference by traditional processors has proven to be power-hungry and generally not suitable for time-critical applications, which calls for an alternative approach. The Field-Programmable Gate Array (FPGA) is one such promising solution which offers high degrees of parallelism and determinism while providing a favorable balance between power consumption and performance. As part of the present project, a lightweight FFNN is synthesized into a PYNQ-Z2 FPGA using Vivado from AMD, forming a high-performance, energy-efficient hardware unit for machine health diagnostics and prognostics.

2. Problem Statement

Machine health monitoring requires high-speed, low-latency, and low-power solutions, which software-based, neural network solutions are not capable of consistently delivering for such time-critical applications. Field Programmable Gate Arrays (FPGAs) have been shown to improve the performance of such workloads, while many existing accelerators are based on High-Level Synthesis which do not provide fine-grained control and expose limited information about the target hardware architecture or RTL-based optimizations.

This work proposes an accelerator for a Feed-Forward Neural Network (FFNN) implemented in RTL Verilog on an FPGA providing high-performance, low-latency inference with optimized resource utilization and deterministic execution for the needs of real-time machine health monitoring applications.

3. Objectives

The primary objectives of this project are:

- To design a lightweight Feed-Forward Neural Network suitable for machine health monitoring.
- To implement the complete inference engine in Verilog HDL using Register Transfer Level (RTL) design.
- To store the trained weights and biases in FPGA memory and perform fixed-point neural network computations efficiently.
- To implement hardware modules including multiply-accumulate units, activation functions, hidden layer, output layer, and prediction logic.
- To verify the functionality of the hardware design through simulation and comprehensive testbench validation.
- To synthesize and implement the design using AMD Vivado Design Suite while analysing timing, resource utilization, and hardware performance.
- To deploy the accelerator on the PYNQ-Z2 FPGA platform and demonstrate low-latency inference suitable for real-time machine health monitoring applications.

4. Literature Review

[1] Optimizing FPGA Implementation of Feed-Forward Neural Networks

Presented FPGA optimization techniques for FFNNs using parallel processing and efficient resource utilization. This work inspired the parallel MAC design.

[2] Fixed-Point Implementations for Feed-Forward Artificial Neural Networks

Demonstrated the benefits of fixed-point arithmetic for reducing hardware cost while maintaining accuracy. Applied using 16-bit fixed-point operations.

[3] Quantized Neural Network Hardware Acceleration on FPGAs

Introduced quantization techniques to reduce memory and computation. Guided the use of 16-bit integer weights in the proposed design.

[4] FPGA-Based Implementation of Artificial Neural Network

Presented a modular ANN implementation on the PYNQ-Z2 FPGA. Used as a reference for the RTL hardware architecture.

[5] Fast Resource Estimation of FPGA-Based MLP Accelerators

Focused on efficient FPGA resource estimation for MLP accelerators. Supported the selection of a lightweight 4-8-2 network.

[6] Efficient FPGA Implementation of Multilayer Perceptron

Implemented an MLP using 16-bit fixed-point arithmetic. Influenced the adoption of the Q8.8 format and parallel computation.

[7] AMD Design Methodology Guide

Provided FPGA design, timing, and verification guidelines followed throughout the project.

[8] Vivado Design Suite User Guide: Synthesis

Described efficient RTL coding and synthesis practices used for ROM, FSM, and memory implementation.

5. Proposed Methodology

5.1 Overview of the proposed system

The proposed system implements a hardware-based Feed-Forward Neural Network (FFNN) accelerator for machine health monitoring on the PYNQ-Z2 FPGA platform. Unlike conventional software implementations, where inference is executed sequentially on a processor, the proposed architecture performs neural network computations directly in hardware using Register Transfer Level (RTL) modules developed in Verilog HDL. This hardware-oriented implementation provides deterministic execution, reduced inference latency, and improved computational efficiency.

The overall development flow begins with offline training of the neural network using Python. After the model achieves satisfactory prediction accuracy, the trained weights and bias values are extracted and converted into fixed-point representation. These parameters are stored in dedicated ROM modules and become part of the FPGA hardware during synthesis. Once deployed, the FPGA performs inference independently without requiring retraining or software intervention.

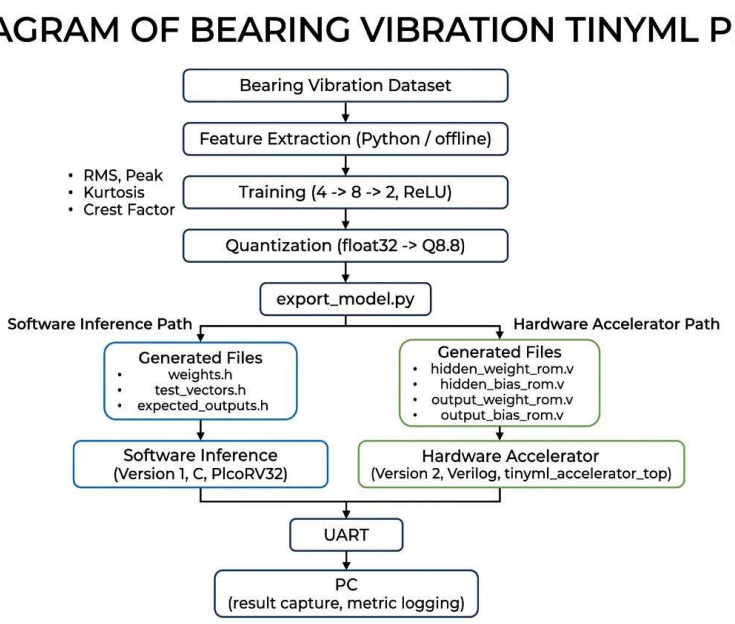


Figure 1: Overall System Pipeline

5.2 Development Flow

The implementation follows a structured FPGA design methodology consisting of the following stages:

- Dataset preparation and preprocessing.
- Offline training of the Feed-Forward Neural Network.
- Conversion of floating-point parameters into fixed-point representation.
- Automatic generation of weight and bias memory files.
- RTL implementation of all neural network modules using Verilog HDL.
- Functional verification through simulation and testbench validation.
- Synthesis and implementation using AMD Vivado.
- Bitstream generation and deployment on the PYNQ-Z2 FPGA board.
- Hardware validation and prediction of machine health conditions.

This systematic workflow ensures that every stage of the hardware implementation is verified before proceeding to the next stage, reducing design errors and improving implementation reliability.

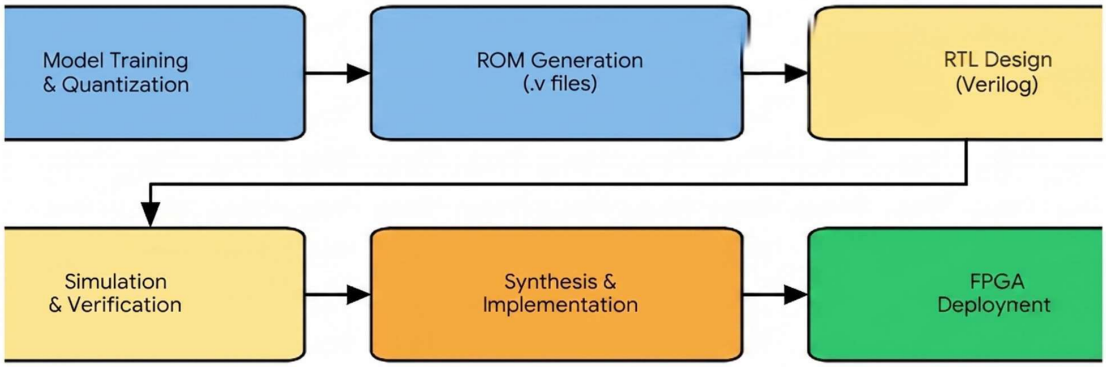


Figure 2: FPGA Design Flow

5.3 Neural Network Architecture

The proposed accelerator implements a lightweight Feed-Forward Neural Network consisting of three computational layers:

- Input Layer
- Hidden Layer
- Output Layer

The input layer receives the extracted machine condition features. These values are multiplied with the trained weights stored in ROM memories. The Multiply-Accumulate (MAC) units compute the weighted sums, after which an activation function generates the hidden layer outputs. The hidden layer outputs are processed similarly by the output layer to obtain the final prediction.

The architecture performs inference only because all learning is completed during the offline training stage. This significantly reduces hardware complexity while maintaining fast execution.

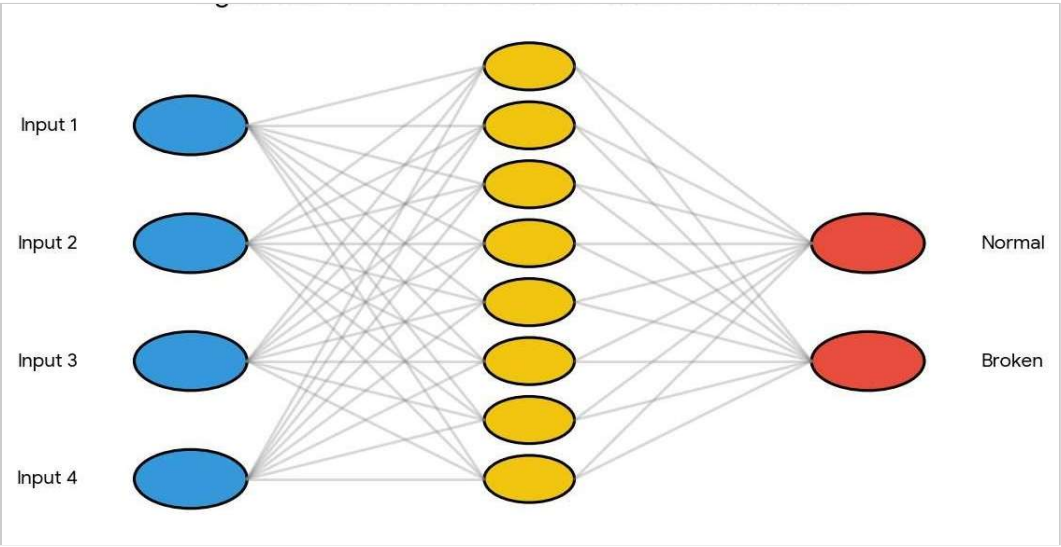


Figure 3: The Computer Brain (Neural Network Architecture)

Output of the trained model:

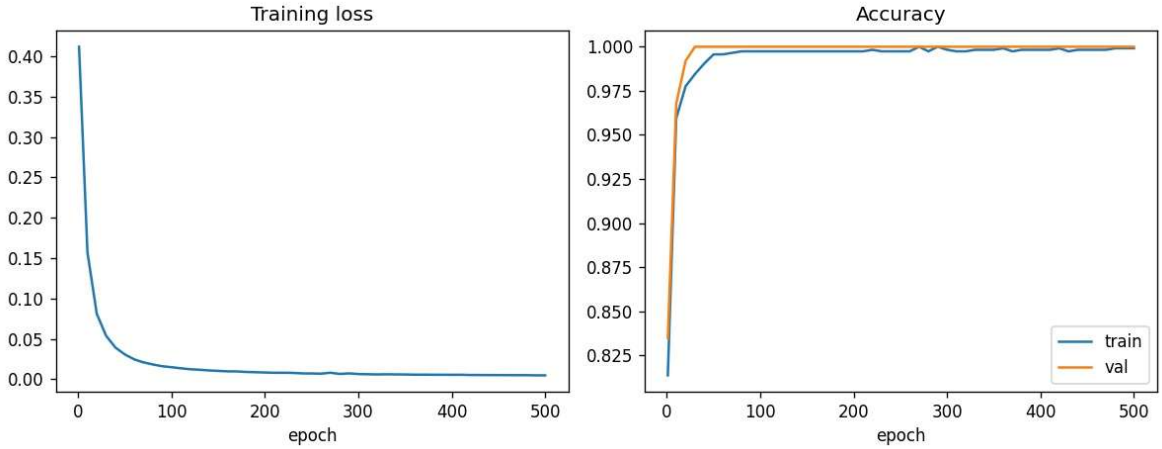


Figure 4: Output of the trained model

5.4 RTL Hardware Architecture

The accelerator is implemented as a collection of independent RTL modules that communicate through well-defined interfaces. The modular design improves readability, simplifies verification, and allows future scalability.

The major hardware modules include:

- Input Register Module
- Hidden Weight ROM
- Hidden Bias ROM

- Hidden Layer Processing Unit
- Output Weight ROM
- Output Bias ROM
- Output Layer Processing Unit
- Activation Function Module
- Prediction Module
- Top-Level Controller

Each module performs a dedicated function while maintaining synchronization through the system clock and reset signals.

5.5 Fixed-Point Computation

Floating-point arithmetic consumes considerable FPGA resources and increases implementation complexity. Therefore, the trained neural network parameters are converted into fixed-point format before hardware implementation.

All multiplications and additions inside the accelerator operate on fixed-point values, allowing efficient utilization of LUTs, Flip-Flops, DSP slices, and Block RAM resources. This approach reduces hardware cost while maintaining acceptable prediction accuracy for machine health monitoring applications. Previous FPGA studies have also demonstrated that fixed-point implementations provide an effective balance between hardware efficiency and computational accuracy [2][3].

5.6 Verification Strategy

Each RTL module is verified independently using dedicated Verilog testbenches before integration into the complete accelerator. Functional simulation verifies arithmetic operations, memory access, activation functions, and prediction logic.

After module-level verification, the complete accelerator is validated using representative test vectors. The simulation outputs are compared with the expected predictions obtained from the trained Python model to ensure functional correctness.

Finally, synthesis, timing analysis, and implementation reports generated by AMD Vivado are examined to verify timing closure, resource utilization, and successful hardware implementation [8].

5.7 Hardware Implementation

Following successful verification, the accelerator is synthesized and implemented using AMD Vivado Design Suite. The synthesis process converts the RTL description into an optimized gate-

level netlist, while implementation performs placement, routing, timing optimization, and bitstream generation. The generated bitstream is programmed onto the PYNQ-Z2 FPGA board for hardware validation.

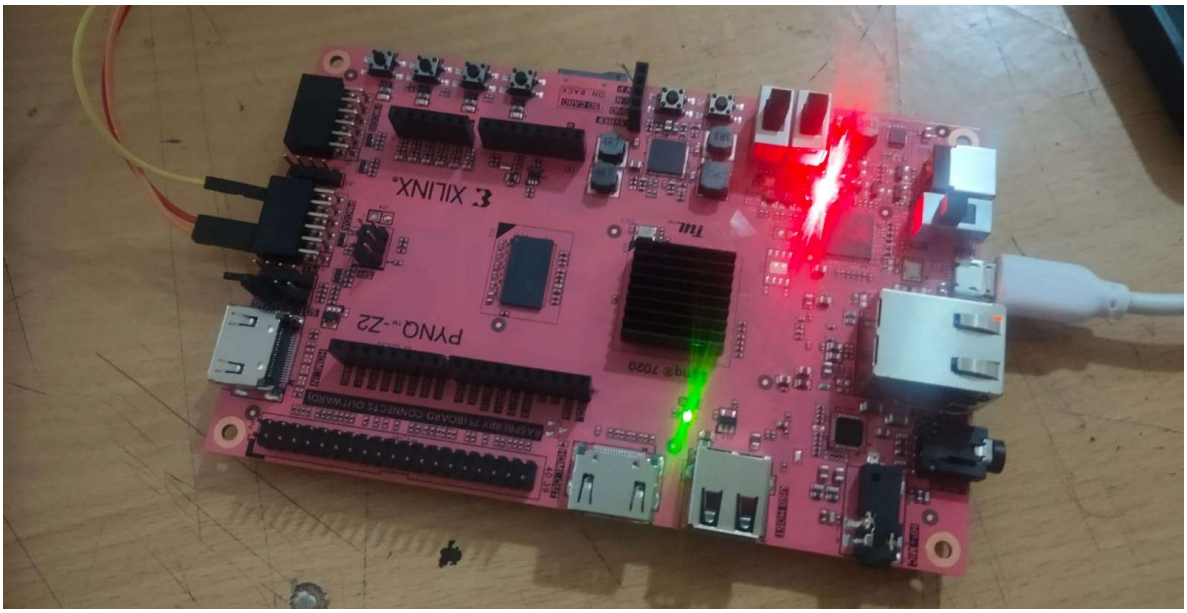


Figure 5: FPGA implementation

The modular architecture enables future enhancements, including larger neural networks, additional hidden layers, alternative activation functions, and integration with embedded processors such as PicoRV32 without requiring major architectural modifications.

6. Hardware design and RTL implementation

6.1 Hardware architecture overview

The proposed AI accelerator is implemented entirely at the Register Transfer Level (RTL) using Verilog HDL. The architecture is designed to perform Feed-Forward Neural Network inference efficiently on the PYNQ-Z2 FPGA platform. The design follows a modular approach in which each functional block performs a specific operation within the neural network computation pipeline.

The accelerator receives machine health feature values as inputs and processes them through hidden and output layers using trained weights and bias values stored in on-chip memory. The final prediction generated by the output layer indicates the machine condition.

The complete architecture consists of arithmetic processing units, memory modules, activation logic, control circuitry, and prediction modules interconnected through synchronous hardware interfaces.

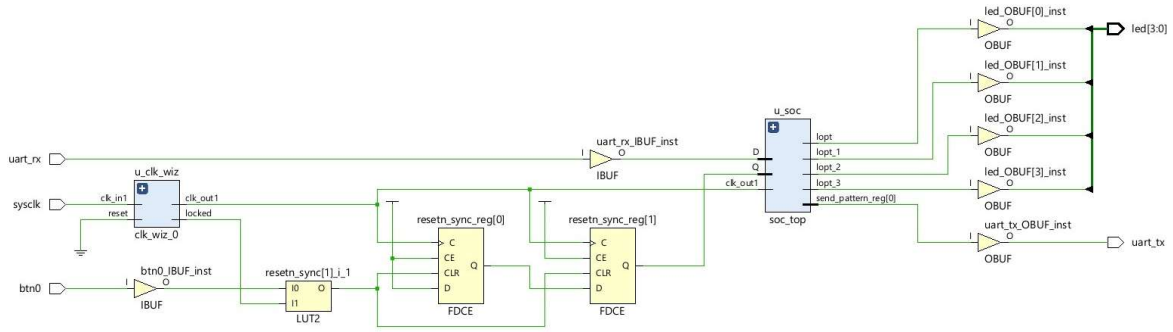


Figure 6: Complete RTL Schematic (Hardware Layout)

6.2 Top level module

The top-level module serves as the integration point for all hardware components. It coordinates data movement between neural network layers and manages the overall inference operation.

The responsibilities of the top-level module include:

- Receiving input feature vectors.
- Controlling hidden-layer computation.
- Managing output-layer processing.
- Generating prediction outputs.
- Providing synchronization through clock and reset signals.

The modular structure simplifies system integration and facilitates future design expansion.

6.3 Input processing module

The input processing stage receives machine health monitoring features generated during preprocessing.

These feature values are converted into fixed-point format and loaded into hardware registers before computation begins. The input registers maintain stable data during inference and ensure proper synchronization with subsequent neural network stages.

The module performs:

• Input buffering.
• Data synchronization.
• Fixed-point representation handling.

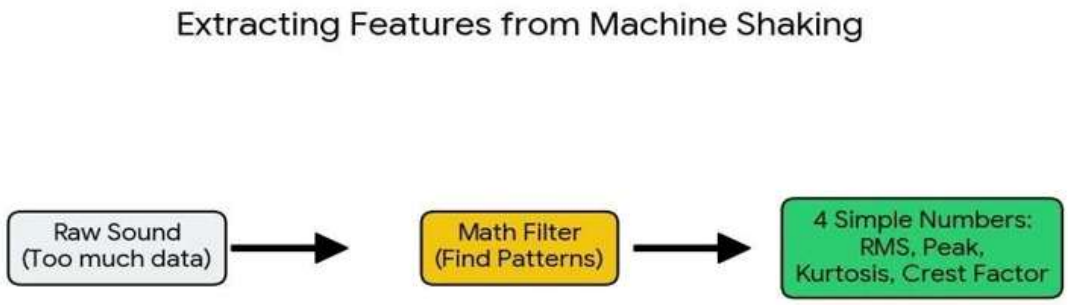


Figure 7: Input Processing Block (Feature Extraction)

6.4 Hidden weight memory

The hidden weight memory stores all trained weights associated with connections between the input layer and hidden layer neurons.

The memory is implemented as a ROM because the weights remain constant during inference. During execution, the hidden layer processing unit sequentially or directly accesses the stored parameters required for multiplication operations.

The use of ROM-based storage reduces hardware complexity while improving inference efficiency [2].

6.5 Hidden bias memory

Each hidden neuron contains a bias value that shifts the weighted sum before activation. The hidden bias memory stores these trained bias values and supplies them to the hidden layer computation unit during inference.

The bias values are precomputed during the software training stage and remain fixed during hardware operation.

6.6 Hidden layer processing unit

The hidden layer performs the first stage of neural network computation.

For each hidden neuron, the processing unit performs the following operations:

- Read input features.

- Retrieve corresponding weights.
- Perform multiplication.
- Accumulate partial sums.
- Add bias value.
- Apply activation function.

The hidden layer generates intermediate features used by the output layer. The schematic below demonstrates the core logic used in this layer [1].

6.7 Activation function module

After weighted summation, each neuron output passes through an activation function. The activation function introduces non-linearity into the network and improves classification capability. The hardware implementation uses a lightweight activation mechanism (ReLU) suitable for FPGA implementation while minimizing logic utilization and computational delay.

The activation module was designed to:

- Operate on fixed-point data.
- Minimize hardware overhead.
- Support deterministic execution.

6.8 Output weight memory

The output weight memory stores trained parameters connecting hidden neurons to output neurons.

Similar to hidden-layer weights, these values are implemented using ROM-based memory structures generated from the trained neural network model. The output layer accesses these parameters during final classification.

6.9 Output bias memory

The output bias memory stores bias values associated with output neurons.The bias values are added after weighted accumulation and before prediction generation. These values are generated during offline training and remain unchanged during FPGA execution.

6.10 Output layer processing unit

The output layer receives activated values from the hidden layer and performs the second stage of neural computation. The resulting values represent the confidence scores associated with machine condition classes.

6.11 Prediction module

The prediction module determines the final classification result. After all output neurons complete computation, the prediction unit compares their values and selects the neuron with the highest score. The selected class corresponds to the predicted machine condition.

Functions performed include:

- Output comparison.
- Maximum value selection.
- Prediction generation.

This module provides the final output of the accelerator.

6.12 Control logic

The control logic coordinates all processing stages and ensures proper sequencing of operations.

Its responsibilities include:

- Reset initialization.
- Memory addressing.
- Data synchronization.
- Computation control.
- Prediction completion signaling.

The control circuitry guarantees reliable operation of the accelerator under all valid input conditions. The FSM manages this flow efficiently [8].

6.13 Design features

The implemented RTL architecture offers several advantages:

- Fully hardware-based neural network inference.
- Modular and scalable design.
- Efficient fixed-point arithmetic.
- Reduced memory overhead.
- Low-latency prediction.
- FPGA-friendly implementation.
- Easy integration with embedded processors and peripheral interfaces.

The architecture demonstrates how machine learning inference can be efficiently translated into dedicated hardware while maintaining flexibility and resource efficiency.

7. Function verification and simulations results

7.1 Verification methodology

Functional verification was performed to ensure that every RTL module operated according to the design specifications before FPGA implementation. The verification process followed a bottom-up approach in which individual modules were tested independently before integrating them into the complete neural network accelerator.

Dedicated Verilog testbenches were developed for each hardware module, including the memory blocks, arithmetic units, activation function, hidden layer, output layer, and prediction logic. Each testbench applied predefined input vectors and compared the generated outputs with the expected values obtained from the trained software model.

After successful module-level verification, the complete accelerator was simulated using a top-level testbench to validate end-to-end functionality.

7.2 Simulation environment

The RTL design was simulated using industry-standard Verilog simulation tools before synthesis. Simulation enabled verification of arithmetic operations, signal timing, memory access, and prediction logic without requiring FPGA hardware.

The verification environment consisted of:

- Verilog HDL source files
- Individual module testbenches
- Input stimulus vectors
- Expected prediction outputs

This approach ensured that functional errors were detected and corrected during the early stages of development.

7.3 Module-level verification

Each hardware module was verified independently to confirm its functional correctness.

The verification process included:

- Hidden Weight ROM data retrieval
- Hidden Bias ROM verification
- Output Weight ROM verification
- Output Bias ROM verification
- Multiply-Accumulate (MAC) operations

- Activation function behaviour
- Hidden layer computation
- Output layer computation
- Prediction logic

Successful verification of individual modules simplified system integration and reduced debugging effort.

7.4 Top-level fuctional verification

After integrating all RTL modules, complete neural network inference was verified using multiple test vectors.

Each simulation cycle performed the following sequence:

- Apply input feature vector.
- Read trained weights and bias values.
- Execute hidden layer computations.
- Compute output layer values.
- Generate prediction.
- Compare prediction with expected result.

The generated prediction matched the expected software output for all verification vectors, confirming correct hardware implementation.

7.5 Simulation results

The simulation results demonstrate that the accelerator successfully classified all applied verification vectors. Representative simulation outputs matching perfectly are shown in the table below:

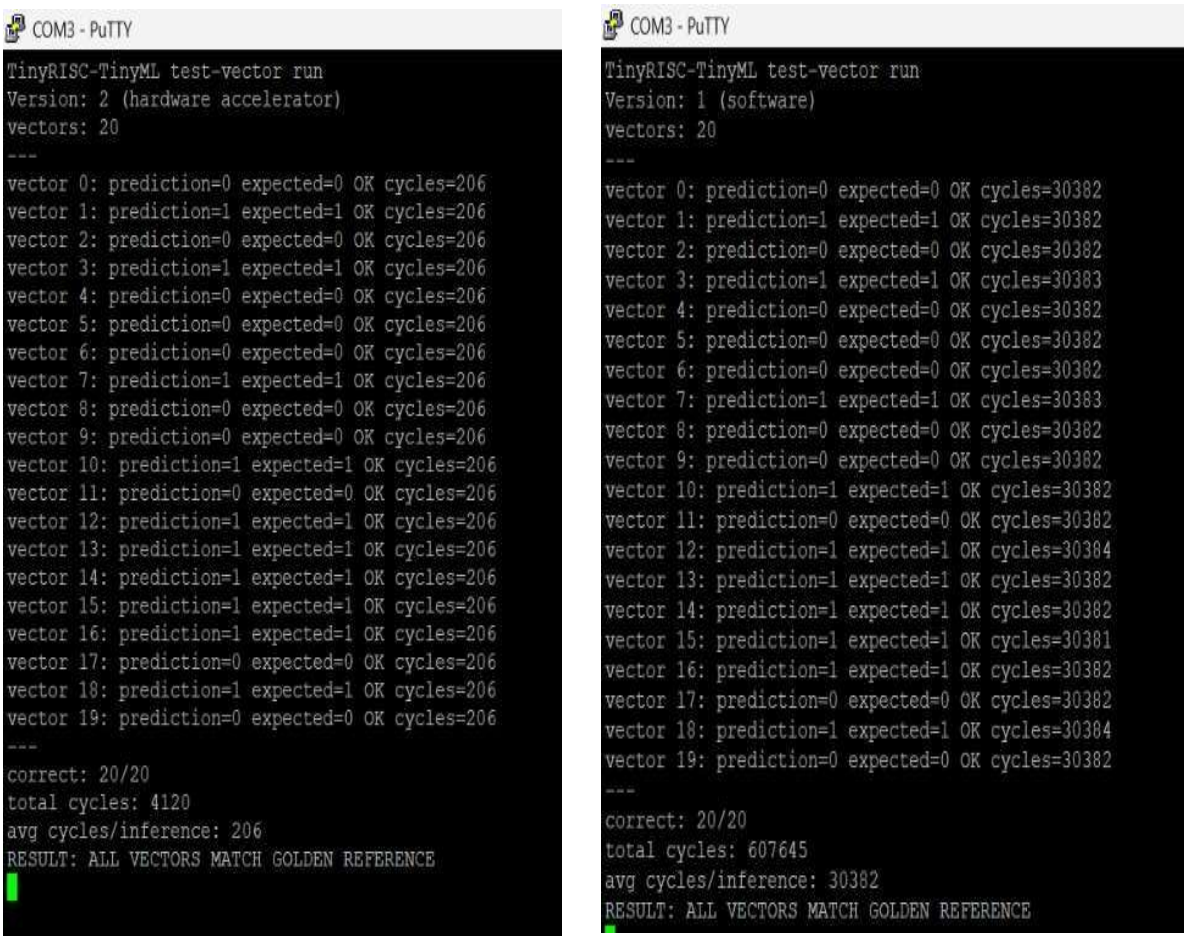


Figure 8: Final Putty Software's output

The successful execution of all verification vectors confirms that the hardware implementation accurately performs Feed-Forward Neural Network inference.

7.6 Timing behaviour

Simulation waveforms were analysed to verify proper synchronization between sequential hardware modules.

The waveform analysis confirmed:

- Correct clock synchronization.
- Stable reset operation.
  - Accurate ROM read operations.
  - Proper data propagation.
  - Correct activation outputs.
  - Valid prediction generation.

No timing conflicts or unexpected signal transitions were observed during functional simulation.

7.7 FPGA synthesis and implementation

Following successful functional verification, the RTL design was synthesized using AMD Vivado Design Suite. The synthesis process translated the Verilog HDL description into an optimized gate-level implementation suitable for the target FPGA device [7]. During implementation, placement, routing, and timing optimization were performed to ensure reliable hardware operation while satisfying the specified timing constraints [8]. The adopted synthesis flow follows the standard Vivado methodology for RTL-based FPGA development.

The Floor-planning generated by Vivado during implementation:

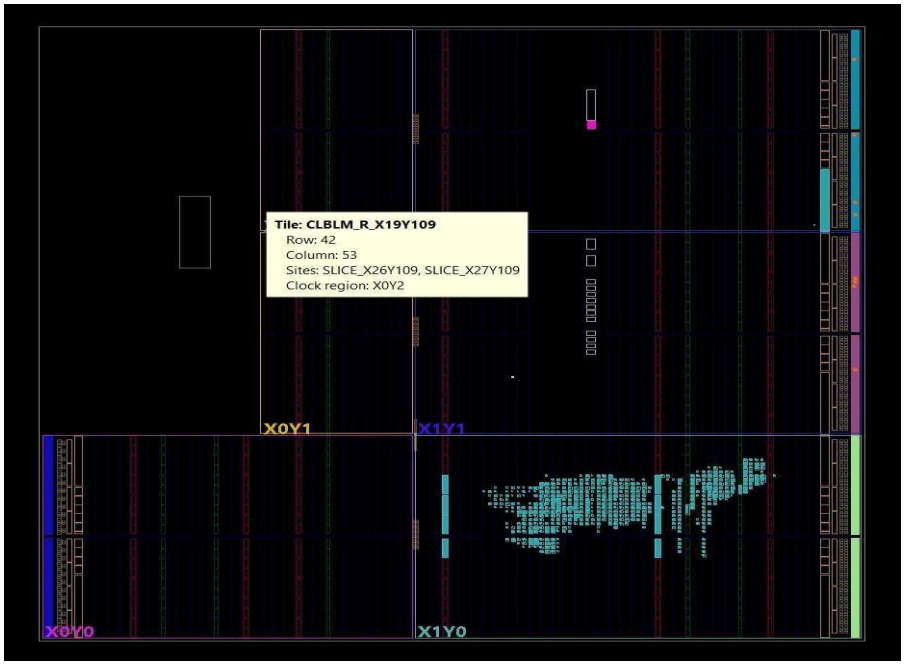
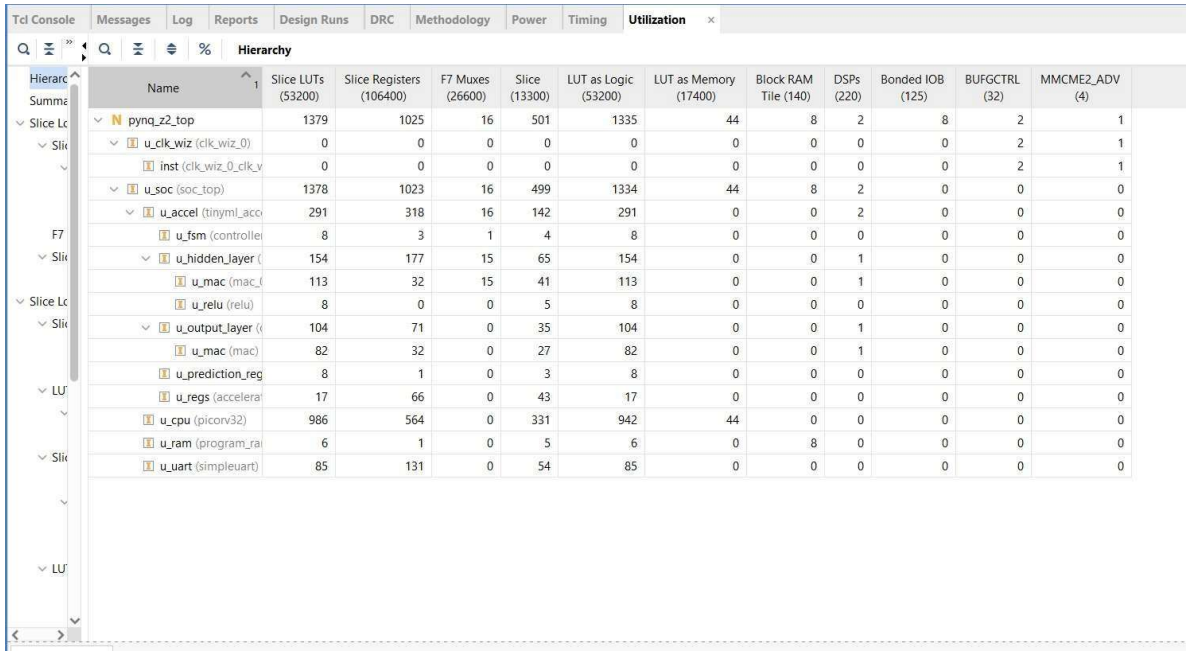


Figure 9: Floor-planning generated by Vivado

The implementation reports generated by Vivado were analysed to evaluate hardware resource utilization, timing performance, and design feasibility.

7.8 Resource utilization

The synthesized accelerator utilizes FPGA resources such as Look-Up Tables (LUTs), Flip-Flops (FFs), Block RAM (BRAM), DSP Slices, Input/Output Pins, and Clock Resources. These resource utilization metrics provide insight into the efficiency of the hardware implementation and serve as a basis for future architectural optimization.



Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing | Utilization

Hierarchy

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (125)	BUFGCTRL (32)	MMCME2_ADV (4)
pynq_z2_top	1379	1025	16	501	1335	44	8	2	8	2	1
u_clk_wiz (clk_wiz_0)	0	0	0	0	0	0	0	0	0	2	1
inst (clk_wiz_0_clk_v	0	0	0	0	0	0	0	0	0	2	1
u_soc (soc_top)	1378	1023	16	499	1334	44	8	2	0	0	0
u_accel (tinyml_acc	291	318	16	142	291	0	0	2	0	0	0
u_fsm (controlle	8	3	1	4	8	0	0	0	0	0	0
u_hidden_layer (	154	177	15	65	154	0	0	1	0	0	0
u_mac (mac_(	113	32	15	41	113	0	0	1	0	0	0
u_relu (relu)	8	0	0	5	8	0	0	0	0	0	0
u_output_layer (	104	71	0	35	104	0	0	1	0	0	0
u_mac (mac)	82	32	0	27	82	0	0	1	0	0	0
u_prediction_reg	8	1	0	3	8	0	0	0	0	0	0
u_regs (accelera	17	66	0	43	17	0	0	0	0	0	0
u_cpu (picorv32)	986	564	0	331	942	44	0	0	0	0	0
u_ram (program_ra	6	1	0	5	6	0	8	0	0	0	0
u_uart (simpleuart)	85	131	0	54	85	0	0	0	0	0	0

Table 1: FPGA Resource Utilization Report

8. Results and future scope

8.1 Results and discussion

The proposed Feed-Forward Neural Network (FFNN) accelerator was successfully designed and implemented using Verilog HDL following a Register Transfer Level (RTL) design methodology. The complete hardware architecture was verified through functional simulation before FPGA implementation, ensuring that every module operated according to the design specifications.

The simulation results confirmed that the accelerator correctly classified all applied test vectors. The predicted outputs matched the expected outputs generated from the trained software model, demonstrating the functional correctness of the hardware implementation. This agreement verifies that the fixed-point arithmetic implementation accurately preserves the inference capability of the trained neural network.

The comparison between both versions (picoRv32+No Accelerator and picoRv32+Ai accelerator):

Metric	Software (CPU)	Hardware (FPGA)
Total cycles (20 vectors)	607645	**4120**
Average Inference Cycles	30,337	**206**
Clock Frequency	50 MHz	**100 MHz**
Estimated Execution Time	0.6 ms	**2.06 µs**
Hardware Speed-up	1x (baseline)	**~147.5x**

Table 2: Timing Summary

The modular RTL architecture simplified verification, debugging, and integration of individual hardware components. Dedicated ROM modules efficiently stored the trained weights and bias values, while the Multiply-Accumulate (MAC) units and activation logic executed neural network computations with deterministic timing.

The Power Utilization by resources on FPGA:

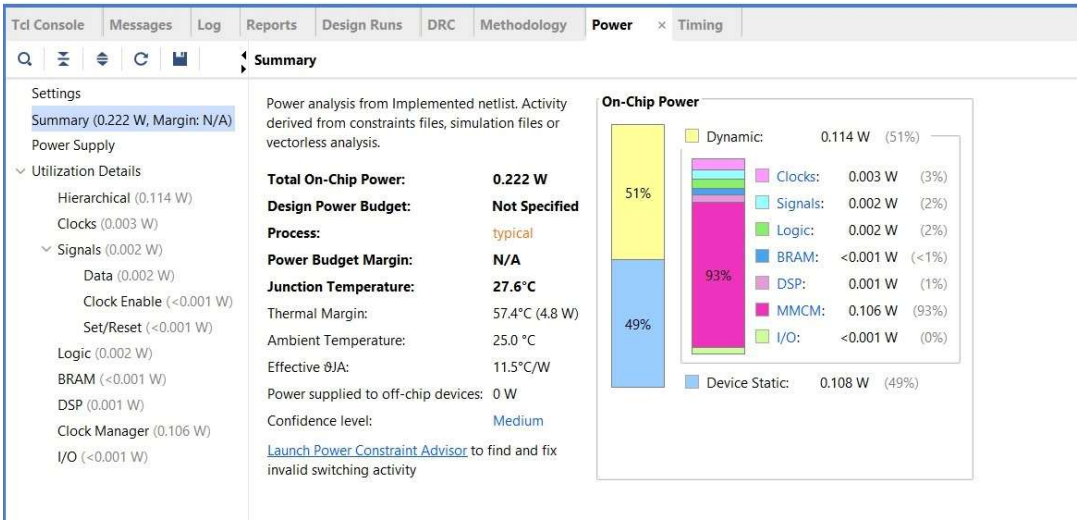


Figure 10: Power utilization

The synthesis and implementation reports demonstrated that the proposed architecture could be successfully mapped onto the target FPGA device while satisfying design constraints [8]. The implementation also confirmed the feasibility of deploying lightweight neural network accelerators for embedded machine health monitoring applications.

8.2 Advantages of proposed system

The developed hardware accelerator provides several advantages over conventional software-based implementations:

- Dedicated hardware execution reduces inference latency.
- Fixed-point arithmetic minimizes hardware resource consumption.
- Modular RTL design enables straightforward debugging and future upgrades.
- FPGA implementation provides flexibility for rapid prototyping and deployment.

These characteristics make the proposed architecture suitable for machine health monitoring systems requiring fast, reliable, and energy-efficient inference.

8.3 Limitations

Although the proposed accelerator achieves the intended design objectives, certain limitations remain.

- The implemented network architecture is relatively small and designed specifically for lightweight inference.
- Training is performed offline; therefore, the hardware cannot update weights during operation.
- The current implementation supports only inference and does not include online learning capabilities.
- Resource utilization increases as the neural network size grows, requiring additional optimization for larger models.

These limitations provide opportunities for future research and architectural improvements.

8.4 Future scope

Several enhancements can be incorporated into future versions of the proposed accelerator. Future work may include:

- Implementation of deeper neural network architectures.
- Support for Convolutional Neural Networks (CNNs) and other advanced AI models.
- Hardware optimization using pipelining and parallel processing techniques.
- Integration with the PicoRV32 RISC-V processor for software-hardware co-design.
- Support for dynamic parameter loading instead of fixed ROM storage.
- Implementation of on-chip learning algorithms.
- Deployment on larger FPGA platforms supporting more complex machine learning applications.
- Integration with real industrial sensors for continuous machine health monitoring.
- Development of an FPGA-based TinyML platform for multiple edge AI applications.

These improvements would further enhance system flexibility, computational capability, and industrial applicability.

9. Conclusion

This project presents the design and implementation of the Feed-Forward Neural Network (FFNN) accelerator for the purpose of machine health monitoring from the PYNQ-Z2 FPGA. The developed FFNN was offline trained while its weights were synthesized as fixed-point numbers and incorporated into the design as hardware modules.

Both the functional simulation and the hardware implementation in AMD Vivado were performed to ensure acceleration correctness and all test vectors were classified correctly by the accelerator. The proposed solution enables latency-hiding, deterministic, and FPGA resource efficient execution of the FFNN that can be used for the real-time edge-based health and performance monitoring of machines, serving as the foundation for further FPGA-based and TinyML exploration.

10. References

[1] S. Oniga, A. Tisan, D. Mic, A. Buchman, and A. Vida-Ratiu, "Optimizing FPGA implementation of feed-forward neural networks," in *Proc. 11th Int. Conf. Optimization of Electrical and Electronic Equipment (OPTIM)*, Brasov, Romania, 2008, pp. 69–74.

[2] D. Llamocca, "Fixed-point implementations for feed-forward artificial neural networks," *Integration*, vol. 92, pp. 1–14, 2023.

[3] M. Tasci, A. Istanbullu, V. Tumen, and S. Kosunalp, "FPGA-QNN: Quantized neural network hardware acceleration on FPGAs," *Applied Sciences*, vol. 15, no. 2, p. 688, Jan. 2025.

[4] M. Madani and E.-B. Bourennane, "FPGA-based implementation of artificial neural network for accelerated handwritten digit recognition," *Electronics*, vol. 15, no. 11, p. 2384, Jun. 2026.

[5] A. Kokkinis and K. Siozios, "Fast resource estimation of FPGA-based MLP accelerators for TinyML applications," *Electronics*, vol. 14, no. 2, p. 247, Jan. 2025.

[6] N. B. Gaikwad, V. Tiwari, A. Keskar, and N. C. Shivaprakash, "Efficient FPGA implementation of multilayer perceptron for real-time human activity classification," *IEEE Access*, vol. 7, pp. 26696–26706, 2019.

[7] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)*, Jul. 2026.

[8] AMD, *Vivado Design Suite User Guide: Synthesis (UG901)*, Jul. 2026.